conftest.py

```python
import pytest
import allure
from pathlib import Path
from playwright.sync_api import sync_playwright

# ======================================================================
# PYTEST + PLAYWRIGHT TEST CONFIGURATION FILE
# ======================================================================
# This file provides:
# 1. Command-line options (browser, base URL, video, screenshots, etc.)
# 2. Hooks to track test results
# 3. Fixtures for browser setup and teardown
# 4. Screenshot, video, and trace attachments to Allure reports
# ======================================================================


# ----------------------------------------------------------------------
# STEP 1: ADD COMMAND LINE OPTIONS
# ----------------------------------------------------------------------
def pytest_addoption(parser):
    """
    Adds command line options for test configuration.
    You can override these when running pytest or store defaults in pytest.ini.
    """
    parser.addoption("--browser", default="chromium", help="Browser: chromium, firefox, webkit")
    parser.addoption("--headed", action="store_true", help="Run in headed (visible) mode")
    parser.addoption("--base-url", default="https://tutorialsninja.com/demo/", help="Base URL
for tests")
    parser.addoption("--video", default="retain-on-failure", help="Record video: on, off,
retain-on-failure")
    parser.addoption("--screenshot", default="only-on-failure", help="Take screenshot: on, off,
only-on-failure")
    parser.addoption("--tracing", default="retain-on-failure", help="Tracing: on, off, retain-
on-failure")


# ----------------------------------------------------------------------
# STEP 2: GET CONFIGURATION VALUE (CMDLINE OR pytest.ini)
# ----------------------------------------------------------------------
def get_config_value(config, option_name):
    """
    Helper to read configuration values.
    Tries to get from command line first, otherwise from pytest.ini.
    Supports both string and boolean options.
    """
    # Try command-line first
    cmd_value = config.getoption(option_name)
    if cmd_value is not None:
        return cmd_value

    # Fallback to pytest.ini
    if option_name == "headed":
        ini_value = config.getini(option_name)
        return ini_value.lower() == "true" if isinstance(ini_value, str) else ini_value
    else:
        return config.getini(option_name)


# ----------------------------------------------------------------------
# STEP 3: HOOK TO TRACK TEST RESULTS (PASS/FAIL)
# ----------------------------------------------------------------------
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    """
    Captures the test result (pass/fail/skip) after each test.
```

```python
    This is used later to decide whether to take screenshots or save traces.
    """
    outcome = yield
    report = outcome.get_result()
    setattr(item, f"rep_{report.when}", report)


# ---------------------------------------------------------------------------
# STEP 4: FIXTURE 1 - BROWSER CONTEXT SETUP
# ---------------------------------------------------------------------------
@pytest.fixture(scope="function")
def browser_context(request):
    """
    Creates and manages the Playwright browser context.
    - Reads configuration (browser, headed mode, video settings)
    - Starts the Playwright browser
    - Enables video recording if configured
    - Cleans up automatically after each test
    """
    # Read configuration values
    browser_name = get_config_value(request.config, "browser")
    headed_flag = get_config_value(request.config, "headed")
    video_option = get_config_value(request.config, "video")

    print(f"🎯 Starting browser: {browser_name}")
    print(f"🎯 Headless mode: {not headed_flag} (headed={headed_flag})")

    # Start Playwright
    playwright = sync_playwright().start()

    # Launch the specified browser
    if browser_name.lower() == "chromium":
        browser = playwright.chromium.launch(headless=not headed_flag)
    elif browser_name.lower() == "firefox":
        browser = playwright.firefox.launch(headless=not headed_flag)
    elif browser_name.lower() == "webkit":
        browser = playwright.webkit.launch(headless=not headed_flag)
    else:
        raise ValueError(f"❌ Unsupported browser: {browser_name}")

    # Create a browser context (optionally with video recording)
    if video_option in ["on", "retain-on-failure"]:
        context = browser.new_context(record_video_dir="reports/videos")
    else:
        context = browser.new_context()

    # Yield the context for use in tests
    yield context

    # Clean up after the test
    print("🧹 Closing browser context and stopping Playwright...")
    context.close()
    browser.close()
    playwright.stop()


# ---------------------------------------------------------------------------
# STEP 5: FIXTURE 2 - PAGE CREATION AND TEST ARTIFACT MANAGEMENT
# ---------------------------------------------------------------------------
@pytest.fixture(scope="function")
def page(request, browser_context):
    """
    Creates a new browser page for each test.
    - Navigates to the base URL
    - Starts tracing (if enabled)
    - Captures screenshots, traces, and videos for failed tests
    - Attaches all artifacts to Allure report
    """
    # Read test configuration
```

```python
base_url = get_config_value(request.config, "base_url")
screenshot_option = get_config_value(request.config, "screenshot")
tracing_option = get_config_value(request.config, "tracing")
video_option = get_config_value(request.config, "video")

print(f"🌐 Navigating to: {base_url}")

# Start tracing if enabled
if tracing_option in ["on", "retain-on-failure"]:
    print("📹 Tracing enabled - capturing screenshots and actions")
    browser_context.tracing.start(screenshots=True, snapshots=True, sources=True)

# Create and navigate to base URL
page = browser_context.new_page()
page.goto(base_url)

# Yield the page to the test
yield page

# ---------------------------------------------------------------------------
# After the test: manage artifacts (screenshots, videos, traces)
# ---------------------------------------------------------------------------
test_name = request.node.name
test_failed = hasattr(request.node, "rep_call") and request.node.rep_call.failed

print(f"📊 Test '{test_name}' result: {'❌ FAILED' if test_failed else '✅ PASSED'}")

# Save and attach trace
if tracing_option in ["on", "retain-on-failure"]:
    trace_path = f"reports/traces/{test_name}_trace.zip"
    browser_context.tracing.stop(path=trace_path)
    print(f"💾 Trace saved: {trace_path}")

    # Attach trace to Allure report if test failed
    # Currently ZIP file is not supporting to attach in allure reports
    # if test_failed:
    #     allure.attach.file(
    #         trace_path,
    #         name=f"{test_name}_trace",
    #         attachment_type=allure.attachment_type.ZIP
    #     )
    #     print("📎 Trace attached to Allure report")

# Take screenshot if test failed
if test_failed and screenshot_option in ["on", "only-on-failure"]:
    screenshot_path = f"reports/screenshots/{test_name}.png"
    page.screenshot(path=screenshot_path)
    print(f"📸 Screenshot saved: {screenshot_path}")

    # Attach to Allure report
    allure.attach.file(
        screenshot_path,
        name=f"{test_name}_screenshot",
        attachment_type=allure.attachment_type.PNG
    )
    print("📎 Screenshot attached to Allure report")

# Attach video if available and test failed
if test_failed and video_option in ["on", "retain-on-failure"]:
    video_path = page.video.path() if page.video else None
    if video_path and Path(video_path).exists():
        allure.attach.file(
            video_path,
            name=f"{test_name}_video",
            attachment_type=allure.attachment_type.WEBM
        )
        print("🎥 Video attached to Allure report")
```

Code Explanation

This code is designed to **set up an automation testing environment** using **Pytest + Playwright**, with built-in support for:

✅ Browser control
✅ Screenshot, video, and trace recording
✅ Reporting integration via **Allure**

1️⃣ Command-line Options (pytest_addoption)

You can run tests with different configurations like:

pytest --browser=firefox --headed --video=on

These options can also come from your pytest.ini file.

2️⃣ Config Value Helper (get_config_value)

This function ensures that your framework checks:

1. Command-line arguments first (highest priority)
2. Then falls back to pytest.ini defaults

So you can either run pytest directly or customize pytest.ini easily.

3️⃣ Test Report Hook (pytest_runtest_makereport)

Tracks the **result** of each test:

- Pass
- Fail
- Skip

Later, this info decides whether to take screenshots or save traces.

4️⃣ Browser Context Fixture (browser_context)

Manages the **Playwright browser**:

- Launches Chromium, Firefox, or WebKit
- Starts in headless or headed mode
- Records video (if enabled)
- Cleans up automatically after tests

Yielding the context allows tests to use it temporarily — once the test is done, cleanup happens automatically.

5️⃣ Page Fixture (page)

Responsible for:

- Opening a **new browser tab (page)**
- Navigating to the **base URL**
- Starting **tracing** (recording actions + screenshots)
- Capturing **screenshots, videos, and traces** when a test fails
- Attaching all artifacts to the **Allure report**

This fixture ensures **every test starts cleanly** and leaves behind helpful debugging evidence if something fails.

📦 Artifacts Saved

Type	Path	Trigger
Screenshot	reports/screenshots/	Only when test fails
Video	reports/videos/	Based on --video setting
Trace	reports/traces/	Based on --tracing setting

⚡ In Short

This setup provides:

- ✅ Clean and reusable browser management
- ✅ Support for all browsers (Chromium, Firefox, WebKit)
- ✅ Configurable video, screenshot, and tracing
- ✅ Allure-ready reporting integration
- ✅ Easy to extend and understand for beginners

pytest.ini

```
# ==========================================
# pytest.ini - Pytest Configuration File
# ==========================================

[pytest]

addopts =
    # ------------------------------
    # General Options
    # ------------------------------
    -v
    --browser=chromium
    --headed
    #--base-url=http://localhost/opencart/upload/
    #--base-url=https://tutorialsninja.com/demo/
    --base-url=https://naveenautomationlabs.com/opencart

    # ------------------------------
    # Captures and Reports (Debugging)
    # ------------------------------
    --video=retain-on-failure
    #--video=on
    --screenshot=only-on-failure
    #--screenshot=on
    --tracing=retain-on-failure
    #--tracing=on
    --html=reports/myreport.html --self-contained-html --capture=tee-sys
    --alluredir=reports/allure-results

    # ------------------------------
    # Parallel Execution
    # ------------------------------
    #-n=1
    # --numprocesses=2

    # ------------------------------
    # Test Grouping (Markers)
    # ------------------------------
    # Use one of the below commands when running pytest:
    # -m "sanity"
    # -m "regression"
    # -m "sanity or regression"
    # -m "sanity and regression"
    # -m "sanity and not regression"
    # -m "regression and not sanity"

    -m "sanity or regression"

    # ------------------------------
    # Flaky Test Handling
    # ------------------------------
    #--reruns 2
    #--reruns-delay 2


# ------------------------------
# Custom Markers
# ------------------------------
markers =
    sanity                        # Marks a sanity test (basic quick check)
    regression                    # Marks a regression test (full end-to-end validation)
```